

Homework 3 - Input/Output and Drawing

Course: CIS 284 - Computer Programming 2 - Fall 2026

Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: September 22 at 11:59pm

Zip all files (java and png) and upload to Canvas. **Make sure each file has your name at the top**

For 2 bonus points, answer the following questions (as a comment on Canvas):

1. Give an estimate of how much time you spent on this assignment.
2. How difficult did you find this assignment?

Your responses will be helpful feedback for refining these assignments in future courses.

Assignment Overview

This assignment is in two parts. In part 1, you will modify your **Stats.java** program from homework 2 to read and write the numbers using files instead of command line arguments. In part 2, you will create a visualization of a shuffling algorithm's performance in order to study its effectiveness

Grading Information

The homework will be graded according to the following rubric. Note that your **Stats.java** submission will only be graded on the specific ability to read and write files, and not on any other requirements related to homework 2. *If you can't complete everything, it is better to submit partial work than miss the deadline.*

Points	Name	Description
30	Effort	Do the programs compile and run? Was a reasonable attempt made?
5	Style	Is the code formatted well? Is it readable? Are variables named well?
15	Part 1A	Was your program properly modified to load the numbers from a text file?
15	Part 1B	Was your program properly modified to save the numbers to a text file?
20	Part 2A	Does the code correctly simulate the shuffles given parameters m and n ? Does it generate the proper array of results?
10	Part 2B	Does the code correctly visualize the 2D array as a grid of shaded squares?
5	Part 2C	Did you properly run the experiment and compare the two algorithms?

Note: Your output does not need to match my examples word for word. Feel free to add additional messages or personalize it, as long as it satisfies the requirements.

1A. Reading Files

In this part, you will modify your `Stats.java` program from homework 2. Your submission for this part will not be graded on the requirements in homework 2, only on the ones below. However, I encourage you to fix any problems from homework 2 to benefit your own learning.

Directions: Remove the part of your code that creates the array of numbers using command line arguments. Instead, when your program starts, it should create an empty array (length 0).

Implement a new command: `load` that does the following:

1. Asks the user to enter a filename.
2. Opens the given file using Scanner.
3. Reads the file according to the file format below.
4. Fills a new array with the values in the file.
5. Prints the number of values read into the array.

Note: If the file does not exist, your program *should not exit*. You should catch the exception and print an error message. Otherwise, **you may assume the file is in the correct format**.

File Format: The provided file should be in the following format

First Line: an integer `n`, giving the number of values. The next `n` lines will have 1 number per line.

Example: (provided `small.txt` file)

```
3
4.5
-2.3
1.15
```

Again, note that the first 3 is the number of lines that will follow.

Example of Part 1A Working:

```
java Stats.java
Welcome!

Please enter a command:
load
Enter a file to load:
small.txt
Loaded 3 numbers

Please enter a command:
print
4.500  -2.300  1.150
```

1B. Writing Files

Add another command, called **save**, that will write the numbers to a file:

1. Ask the user for a filename
2. Write the array of numbers to the file (using the correct file format)
3. Print a success message saying how many numbers were written

Example of Part 1B:

```
Please enter a command:
save
Enter a file to load:
values.txt
Successfully wrote 3 numbers
```

Tip: Your **load** command should be able to read a file written by your **save** command.

2. Shuffle Algorithm Visualization

In this activity, you will create a tool that visualizes the effectiveness of a shuffling algorithm. Your program `ShuffleTest.java` depends on 2 parameters:

1. `int m` - the size of array to shuffle
2. `int n` - the number of trials to run

The program will create a $m \times m$ two-dimensional array of integers that keeps track of the results. In this table, row `i` column `j` would count the number of times that value `i` ended up in position `j` after shuffling.

The program will then visualize this data by drawing a grid of shaded squares to indicate frequencies. Better shuffling algorithms should result in a more uniformly shaded grid.

2A. Simulating the Shuffles

In this first part, you will generate the data by running `n` trials.

1. Your program should set variables `m` and `n` using command line arguments. If no command line arguments are given, set them to default values of your choosing.
2. Initialize an 2D `int` array of size $m \times m$. This will record the results of the simulation.
3. Write a loop that runs `n` times. Each time (each trial):
 - Create an array of length `m`, and set each element equal to its index. For example, if `m=4`, then you would create an array `[0,1,2,3]`
 - Shuffle this array using the `shuffle1(arr);` function in the provided code.
 - Record the results in the results array. If value `i` ended up at index `j`, then you would increment the number at row `i`, col `j`. For example, if the shuffle resulted in `[3,1,2,0]`, then you would increment the value in row 3, col 0 (because 3 ended up at index 0).
4. Once you have run `n` trials, you should have collected all the results. Print them out using nested for loops. You should format the results nicely so the columns are aligned:

Here is what your output should look like for `m=10` and `n=10000`. For a good shuffling algorithm, the counts should average around m/n (here 1000).

```
1007  980 1049 1014  996  978  999 1059  936  982
1018 1056 1026  981  969  993 1036  922 1001  998
1013 1021  965 1020  962  954 1011 1006 1063  985
 959 1022 1025  956 1061 1007  966  974 1008 1022
1037  989 1013 1006 1045  968  996 1016  988  942
 970 1035 1015 1023 1005  969  960  991  971 1061
 960  954  976 1035  958 1008 1031 1036 1001 1041
1038  987  973  963 1033 1047  997 1009  994  959
1023 1004  991  981  987 1037  993  962 1008 1014
 975  952  967 1021  984 1039 1011 1025 1030  996
```

2B. Visualizing the Results

Once part 1A is working, use the **StdDraw** library to draw the results to a GUI window as a 2D grid of shaded squares. It should be a $m \times m$ grid, where each square is shaded according to its value in the results array.



Hint: refer to examples in class for some help drawing grids. To select a shade, you can set the pen color using a **setPenColor(r, g, b)**, where each rgb value is an integer from 0-255. To make a shade of gray, all 3 values should be equal.

Think about how to transform your results to a shade of gray. Since each result should average m/n , then that should translate to a shade of 128. Use that as your scaling factor.

2C. Comparing Shuffles

Run your program analyzing **shuffle1**, and once it is visualizing the data correctly, save the image to file as **shuffle1.png**.

Then, modify your program so it instead calls **shuffle2**, and again save the image as **shuffle2.png**

At the top of **ShuffleTest.java**, write a comment describing the differences you see between the two results for shuffle 1 vs 2. Then say what this tells you about the effectiveness of each shuffle algorithm. Make sure to explain your answer.

Note: include both images in your submission.

2D. Optional Challenge Problem (Extra Credit)

There is also a file **RiffleTest.java** that has a **riffleShuffle** function in the code that simulates a riffle shuffle used when shuffling cards. It will split the array into two parts, then merge them together.



Obviously 1 riffle shuffle will not mix up the cards very well. You will look at how many shuffles do you need to mix up the array properly.

Copy over the code from **ShuffleTest** and modify it so it does the following:

1. Use fixed **m=52** and **n=10000**
2. Run the whole experiment described in parts 2A-2C, and draw the results. For each of the **n** trials, call **riffleShuffle(arr)** 1 time.
3. Pause for 2 seconds **StdDraw.pause(2000)**
4. Repeat steps 1-3, but now do 2 riffle shuffles each trial and draw the results.
5. Continue repeating the outer loop, each time increasing the number of shuffles per trial.

The result should be an animation of the shuffling results using 1 riffle shuffle each trial, then 2, and so on.

How many times would you say you need to riffle shuffle a deck of cards before it is evenly mixed?

2E. Optional Challenge Problem (Extra Credit)

Try coming up with another shuffling method, and test it to see how it performs.